

```
import pandas as pd
import numpy as np
```

```
dataset = pd.read_csv("salary.csv")
dataset
```

	age	education.num	capital.gain	hours.per.week	income
0	90	9	0	40	<=50K
1	82	9	0	18	<=50K
2	66	10	0	40	<=50K
3	54	4	0	40	<=50K
4	41	10	0	40	<=50K
...	...	...	...	...	...
32556	22	10	0	40	<=50K
32557	27	12	0	38	<=50K
32558	40	9	0	40	>50K
32559	58	9	0	40	<=50K
32560	22	9	0	20	<=50K

32561 rows × 5 columns

Next steps: [Generate code with dataset](#) [New interactive sheet](#)

```
print(dataset.shape)
print(dataset.tail())
```

```
(32561, 5)
<bound method NDFrame.tail of
0    90      9      0      40  <=50K
1    82      9      0      18  <=50K
2    66     10      0      40  <=50K
3    54      4      0      40  <=50K
4    41     10      0      40  <=50K
...  ...    ...    ...    ...    ...
32556 22     10      0      40  <=50K
32557 27     12      0      38  <=50K
32558 40      9      0      40  >50K
32559 58      9      0      40  <=50K
32560 22      9      0      20  <=50K
```

[32561 rows x 5 columns]>

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 5 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   age                 32561 non-null  int64
1   education.num       32561 non-null  int64
2   capital.gain        32561 non-null  int64
3   hours.per.week      32561 non-null  int64
4   income              32561 non-null  object
dtypes: int64(4), object(1)
memory usage: 1.2+ MB
```

```
income_set = set(dataset['income'])
dataset['income'] = dataset['income'].map({'<=50K': 0, '>50K': 1}).astype(int)
print(dataset.head())
```

	age	education.num	capital.gain	hours.per.week	income
0	90	9	0	40	0
1	82	9	0	18	0
2	66	10	0	40	0
3	54	4	0	40	0
4	41	10	0	40	0

```
X = dataset.iloc[:, :-1].values #input
X
```

```
array([[90,  9,  0, 40],
       [82,  9,  0, 18],
       [66, 10,  0, 40],
       ...,
       [40,  9,  0, 40],
       [58,  9,  0, 40],
       [22,  9,  0, 20]])
```

```
Y = dataset.iloc[:, -1].values
Y
```

```
array([0, 0, 0, ..., 1, 0, 0])
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,Y,test_size =0.25,random_state = 0)
```

```
X_train
```

```
array([[ 55,   3,   0,  40],
       [ 42,   9,   0,  40],
       [ 42,   9,   0,  40],
       ...,
       [ 26,   8,   0,  51],
       [ 44,  14,   0,  40],
       [ 39,   9, 7298,  40]])
```

```
X_test
```

```
array([[56,  5,  0, 40],
       [19,  9,  0, 25],
       [23, 10,  0, 35],
       ...,
       [44, 10,  0, 55],
       [58,  9,  0, 50],
       [55, 10,  0, 45]])
```

```
y_train
```

```
array([0, 0, 0, ..., 0, 1, 1])
```

```
y_test
```

```
array([0, 0, 0, ..., 0, 0, 0])
```

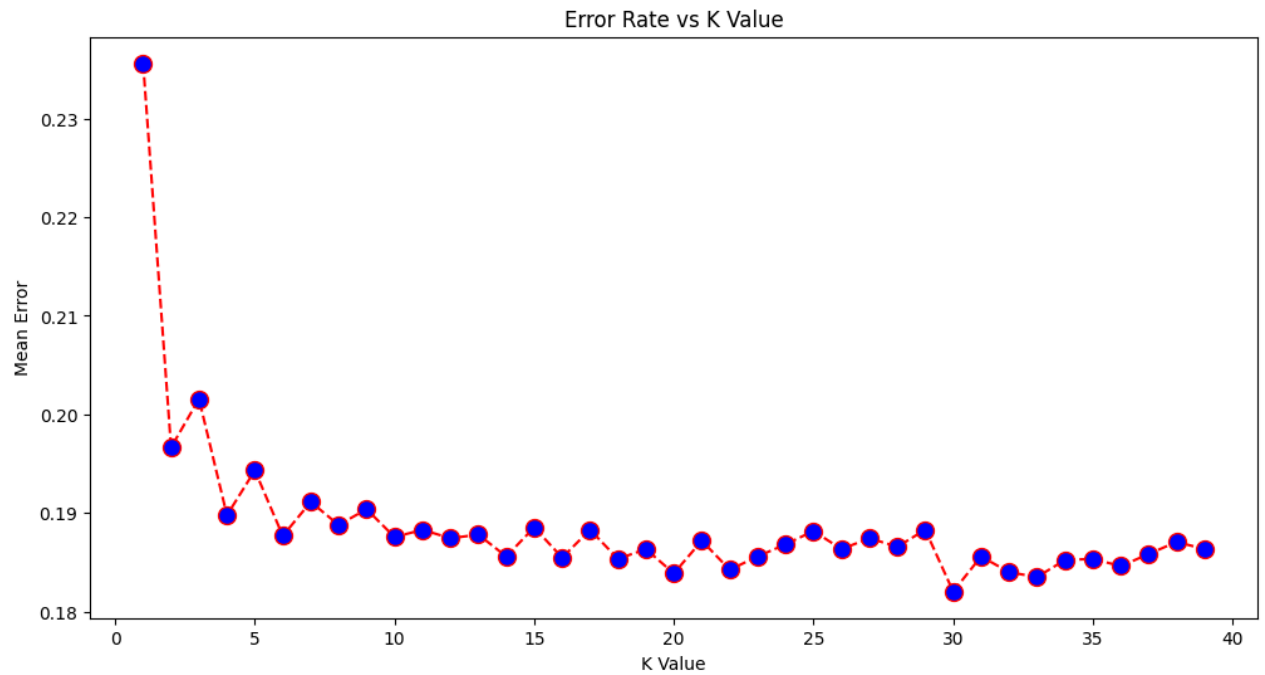
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
import numpy as np

error = []
for i in range(1, 40):
    model = KNeighborsClassifier(n_neighbors=i)
    model.fit(X_train, y_train)
    pred_i = model.predict(X_test)
    error.append(np.mean(pred_i != y_test))

plt.figure(figsize=(12, 6))
plt.plot(range(1, 40), error,
         color='red',
         linestyle='dashed',
         marker='o',
         markerfacecolor='blue',
         markersize=10)
plt.title('Error Rate vs K Value')
plt.xlabel('K Value')
plt.ylabel('Mean Error')
plt.show()

model = KNeighborsClassifier(n_neighbors=2, metric='minkowski', p=2)
model.fit(X_train, y_train)
print(model)
```



KNeighborsClassifier(n_neighbors=2)

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors = 2, metric = 'minkowski', p = 2)
model.fit(X_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(n_neighbors=2)

```
age = int(input("Enter New Employee's Age: "))
edu = int(input("Enter New Employee's Education: "))
cg = int(input("Enter New Employee's Captital Gain: "))
wh = int(input("Enter New Employee's Hour's Per week: "))
newEmp = [[age,edu,cg,wh]]
result = model.predict(sc.transform(newEmp))
print(result)
```

```
if result == 1:
    print("Employee might got Salary above 50K")
else:
    print("Customer might not got Salary above 50K")
```

```
Enter New Employee's Age: 19
Enter New Employee's Education: 12
Enter New Employee's Captital Gain: 240
Enter New Employee's Hour's Per week: 100
[0]
Customer might not got Salary above 50K
```

```
y_pred = model.predict(X_test)
print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
```

```
[[0 0]
 [0 0]
 [0 0]
 ...
 [0 0]
 [0 0]
 [0 0]]
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix: ")
print(cm)

print("Accuracy of the Model: {}%".format(accuracy_score(y_test, y_pred)*100))
```

```
Confusion Matrix:
[[5918 275]
 [1326 622]]
Accuracy of the Model: 80.33411128853949%
```